Veltra – Complete Software Architecture & Implementation Plan

Executive Summary

After extensive research and team discussions, we have finalized the complete software stack for Veltra. The decisions below represent the optimal balance of rapid development, competition-readiness, and real-world viability for our industrial heat safety platform.

📋 Final Technology Stack Decision

Category	Chosen Technology	Rationale
Mobile Framework	Flutter	Cross-platform (iOS + Android) from a single codebase. Perfect for NFC support via `flutter_nfc_kit`.
Backend & Infrastructure	Firebase	All-in-one solution: Authentication, Firestore database, Hosting, Storage, Analytics, Cloud Functions. No server maintenance. Fastest path to MVP.
Local Database	SQLite (`sqflite`)	Offline-first design. Scans stored locally, synced to cloud when online.
On-Device ML	TensorFlow Lite (`tflite_flutter`)	XGBoost model runs locally. No internet required. <50ms inference time.
Voice Recognition	Vosk (`vosk_flutter`)	Fully offline speech-to-text. Works in noisy construction environments.
Cloud Dashboard	React + Firebase Hosting	Modern, responsive UI for safety directors. Hosted on Firebase for zero-config deployment.
State Management	Riverpod	Flexible, testable, works seamlessly with Firebase.
HTTP Client	Dio	For any external API calls (weather/WBGT data).

🧪 Testing & Development Strategy

Development Environment

Component	Setup
Code Editor	Cursor (AI-assisted)
Machine	2018 Intel MacBook Pro (macOS 14/15)
iOS Simulator	Xcode Simulator (for UI/UX testing only)
Android Emulator	Android Studio Emulator
Physical Device	iPhone 17 Pro (for NFC testing)

Testing Approach

Test Phase	What's Tested	How
1. UI/UX Testing	Layout, navigation, responsiveness	iOS Simulator + Android Emulator
2. NFC Testing	Tag reading, data parsing, error handling	Physical iPhone 17 Pro (NFC not supported on simulators)
3. Voice Recognition	Offline speech-to-text accuracy	Physical device in noisy environment simulation
4. ML Inference	XGBoost model performance	All platforms (simulator + physical)
5. Firebase Integration	Auth, Firestore reads/writes, offline sync	All platforms with Firestore emulator
6. End-to-End Flow	Full scan → parse → infer → display → save	Physical device with mock hardware

iOS Development Notes

Item	Status
Apple Developer Account	Free (for testing)
7-Day Expiration	Acceptable for MVP and competitions

NFC Entitlement	Must be added to `ios/Runner/Info.plist`
Deployment Target	iOS 14.0+

📱 App Flow & User Experience

User Roles

Role	Count	Responsibilities
Supervisor / Foreman	1 per site	Set up crew, run scans, view results
Worker	Many	Wear hard hat. Zero interaction with app.

Onboarding Flow

1. Foreman opens app.
2. Signs up with email/password (Firebase Auth).
3. Creates first crew: adds worker names (and optionally assigns NFC IDs).
4. Ready to scan.

> **Note:** No worker accounts. No worker app. Zero friction for workers.

Bottom Navigation (3 Tabs)

Tab	Icon	Purpose
Scan	📷 *(center, primary)*	Main scanning interface. Big prominent button.
Crew	👷	View all workers, their status, scan history.
Settings	⚙️	Account, crew management, privacy policy, export data.

Scan Flow (The Core Experience)

Step	Action	Duration
1	Foreman taps "Scan" button	< 1s

2	Phone starts listening for NFC	Instant
3	Foreman holds phone near hard hat rear	1–2s
4	NFC reads 24-byte payload (6 memory blocks)	500–800ms
5	App validates checksum + continuity loop	< 10ms
6	Worker identified (auto NFC → or manual selection/voice)	< 1s
7	XGBoost model runs locally on 8D feature vector	< 50ms
8	Status displays: GREEN/AMBER/RED with action text	Instant
9	Foreman taps "Log & Next"	< 1s
10	Scan saved locally (SQLite), syncs to cloud if online	< 100ms

UI Screen Breakdown

Results Screen Components

- **Large Color Banner:** GREEN checkmark / AMBER warning / RED alert
- **Action Text:** "Continue work" / "Drink 500mL water" / "Rest 15 min in shade"
- **Worker Name:** Displayed prominently with photo (optional)
- **Metrics (Collapsible):** Skin temperature, sweat rate, salinity shift, Z-score
- **Trend Indicator:** Up/down arrows showing changes from last scan
- **"Log & Next" Button:** Saves result, returns to scan-ready state
- **"Alert" Button:** Send SMS/text to safety director (RED status only)

Crew Tab Features

- **Worker List:** All workers with most recent status (color-coded)
- **Status Filter:** Show GREEN/AMBER/RED only
- **Worker Detail:** Tap to see full scan history + trend graphs
- **Add/Remove Worker:** Manage crew roster

Settings Tab Features

- **Account Management:** Logout, change password
- **Crew Management:** Batch import/export workers
- **Privacy Policy:** View and acknowledge
- **Export Data:** CSV/PDF export of all scans
- **Offline Mode Indicator:** Shows current sync status
- **Version Number:** Build info

Level-Up Features (For Competition Edge)

Feature	Why It Impresses
Offline-First Design	Shows you understand construction site realities
Haptic Feedback	Phone vibrates on successful NFC read—confirmation without looking
Voice Confirmation	App speaks *"Jake, GREEN"* out loud in noisy environments
Trend View	Line graphs showing skin temperature, sweat rate, and Z-score over time for each worker
Share Result	One-tap share scan result as image (for safety documentation)
Dark Mode	Essential for early morning and evening scans
1-Click OSHA Report	Generate PDF audit package from any worker's scan history

Visual Design Principles

Principle	Implementation
High Contrast	Dark background (`#1E1E1E`), white text, bright status colors
Large Touch Targets	Minimum 48px buttons (glove-friendly)
Industrial Aesthetic	Bold typography (Inter or Roboto), sharp corners, subtle textures
Color Coding	**GREEN:** `#00C853`, **AMBER:** `#FFAB00`, **RED:** `#D50000`
Confidence Building	Brief *"✅ Data read"* toast after each scan

🧠 ML Model Training Pipeline

Input Features (8-Dimensional Vector)

#	Feature	Source	Calculation / Method

1	Current forehead skin temperature	T4 Thermistor	Direct reading via ADC
2	Rate of skin temperature change	Calculated	`(T_current - T_prev) / time_elapsed`
3	Current sweat conductance/salinity shift	E5 Conductivity cell	Measured via microamp AC current
4	Current sweat accumulation rate	F3 Impedance ladder	Volume change over time
5	Ambient WBGT	Weather API	Cached, updated on sync
6	Cumulative elapsed shift time	System clock	Hours since shift start
7	Personal salinity Z-score deviation	Database	`(current - baseline_mean) / baseline_std`
8	Acclimatization factor	Database	1.0–2.0 scale based on days worked

Output Classes

Status	Risk Score	Action
GREEN	< 0.35	Continue standard work rotation
AMBER	0.35 – 0.70	Drink 500mL electrolyte mix, re-scan in 30 min
RED	> 0.70	Mandatory 15-min rest in shade, monitor for symptoms

Training Data Sources

Phase	Data Source	Description
MVP (Now)	Synthetic Data Generator	Physics-based thermoregulation model (JOS-3) + realistic noise. Generates 5,000–10,000 labeled samples.
Validation	Public Datasets	EPA Wearables Study, Firefighter Heat Stress Dataset
Future	Real Field Data	Collected from athletic trials and contractor pilot deployments

Synthetic Data Generation Approach

1. Simulate baseline worker physiology (skin temp 36–37°C, sweat rate 0.3–0.8 L/hr).
2. Model environmental conditions (WBGT 20–35°C).
3. Simulate exertion over 8-hour shift (escalating workload).
4. Apply physics-based thermoregulation to generate skin temp and sweat rate.
5. Add realistic sensor noise.
6. Label based on thresholds:
    - **RED:** Skin temp > 38.5°C **OR** Salinity Z > 2.5
    - **AMBER:** Skin temp > 37.5°C **OR** Salinity Z > 1.5
    - **GREEN:** Otherwise

Model Training & Export

Step	Tool / Library	Purpose
1	Python + Pandas	Load synthetic data
2	scikit-learn	Train/test split, preprocessing
3	XGBoost (`xgboost`)	Train gradient-boosted decision tree model
4	XGBoost → TFLite converter	Export to `model.tflite`
5	`tflite_flutter`	Run inference on device

Cold Start Strategy (Shifts 1–4)

- **First 4 Shifts (Hybrid Rule Engine):**
    - WBGT environmental thresholds
    - Population-average physiological thresholds
    - Conservative safety margins
- **Shift 5+ (Personalized Model):**
    - Calculates personalized mean (μ) and standard deviation (σ) for each worker
    - Converts raw readings to Z-scores
    - Uses personalized XGBoost model for inference

🔄 Complete Data Flow

```
[Foreman taps phone on hard hat]
        ↓
[flutter_nfc_kit reads 24 bytes from memory blocks 0x00-0x05]
```

```
       ↓
[App validates checksum + continuity loop]
       ↓
[App parses raw ADC values → physical metrics]
       ↓
[App fetches worker baseline from SQLite]
       ↓
[App fetches cached WBGT (or uses last known value)]
       ↓
[App constructs 8D feature vector]
       ↓
[tflite_flutter runs XGBoost inference → risk score]
       ↓
[App displays GREEN/AMBER/RED + action text]
       ↓
[App saves scan to SQLite]
       ↓
[If online: sync to Firestore]
       ↓
[Cloud dashboard updates live]
       ↓
[Safety director can view trends + 1-click OSHA report]
```

📦 Complete Software Ecosystem Summary

Layer	Technology	Purpose
Mobile Frontend	Flutter	Cross-platform app for foremen
NFC Reading	`flutter_nfc_kit`	Read ISO/IEC 15693 tags
Voice Input	`vosk_flutter`	Offline speech-to-text
Local Storage	`sqflite` (SQLite)	Offline scan history + worker profiles
On-Device ML	`tflite_flutter`	XGBoost model inference
State Management	Riverpod	App state + Firebase integration

Backend	Firebase	Auth, Firestore, Hosting, Storage
Dashboard	React + Firebase Hosting	Safety director web interface
Analytics	Firebase Analytics	User behavior + app performance
Push Notifications	Firebase Cloud Messaging	RED status alerts
API Calls	Dio	Weather API (WBGT) + external services

📊 Database Schema (Firestore)

`Workers` Collection

JSON

```
None
{
  "id": "uuid",
  "name": "string",
  "employee_id": "string",
  "role": "string",
  "site_id": "uuid",
  "nfc_id": "string",
  "photo_url": "string (optional)",
  "created_at": "timestamp"
}
```

`Scans` Collection

JSON

```
None
{
  "id": "uuid",
  "worker_id": "uuid (reference)",
  "timestamp": "timestamp",
  "skin_temp": 37.2,
  "sweat_conductance": 62.0,
  "sweat_volume": 1.2,
```

```
  "sweat_rate": 1.2,
  "wbgt": 28.5,
  "status": "AMBER",
  "recommendation": "Drink 500mL water",
  "z_score": 2.1,
  "synced_at": "timestamp"
}
```

`Baselines` Collection (per worker)

JSON

```
None
{
  "worker_id": "uuid",
  "mu_salinity": 45.0,
  "sigma_salinity": 8.5,
  "mu_temp": 36.8,
  "sigma_temp": 0.4,
  "shift_count": 12,
  "updated_at": "timestamp"
}
```

✅ Scope Matrix

Covered (In Scope)

- NFC tag reading and parsing
- Offline-first data storage (SQLite)
- On-device ML inference (XGBoost via TFLite)
- Voice input (offline Vosk)
- Firebase integration (Auth, Firestore, Storage, Hosting)
- Worker management (add/remove/view)
- Scan history and trend view
- OSHA report PDF generation
- Haptic feedback
- Dark mode
- Sync status indicator

Not Covered (Future Roadmap)

- Real-time continuous monitoring (not part of MVP)
- Multi-site management (can be added later)
- Worker mobile app (intentionally excluded)
- In-app purchases/subscriptions
- Push notifications beyond RED status alerts
- Biometric authentication (fingerprint/face ID)

🚀 Quick Start Commands

1. Flutter Project Setup

Bash

```
None
flutter create veltra
cd veltra
flutter pub add flutter_nfc_kit vosk_flutter sqflite tflite_flutter
riverpod dio firebase_core firebase_auth cloud_firestore
firebase_storage
```

2. Firebase Setup

Bash

```
None
# Install Firebase CLI
npm install -g firebase-tools

# Login to Firebase
firebase login

# Initialize Firebase project (Select: Firestore, Hosting, Storage,
Functions)
firebase init

# Deploy dashboard
firebase deploy --only hosting
```

3. Run App

Bash

```
None
# iOS Simulator
open -a Simulator
flutter run

# Physical iPhone (connected via USB)
flutter run

# Android Emulator (open Android Studio first)
flutter run
```

🏆 Competition Pitch Points

Angle	What to Emphasize
Technical Depth	Full-stack architecture with offline ML, NFC, and real-time sync
Real-World Problem	40 deaths/year + $285,000 insurance penalty per incident
Engineering Trade-offs	Why XGBoost over LLM, why Firebase over AWS, why offline-first
User-Centric Design	Zero friction for workers, 1-second scans, haptic feedback
Innovation	NFC-through-hard-hat, battery-less sensors, on-device ML
Scalability	Firebase handles scale, 77.2% gross margin model